西南交通大學
Southwest Jiaotong University

云计算技术

第3讲 云原生基础与微服务

授课教师：戴朋林

课程代码：SCAI004712

2026年3月



内容大纲

西南交通大学 Southwest Jiaotong University

Docker容器

封装与隔离

（namespace、cgroup）

复杂应用怎么放到容器？

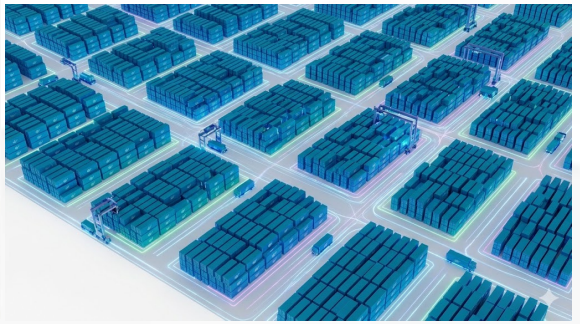
例如：淘宝、微信

微服务与调度管理

拆分、协同、编排







内容大纲

1. 云原生的概念与核心理念
   - CNCF定义与四大要素
   - 云原生与传统开发模式对比
   - 十二要素应用（12Factor App）
2. 微服务架构设计原则
   - 微服务定义与特征
   - 服务拆分策略
   - 通信方式（REST / gRPC / 消息队列）
   - 服务发现与注册、负载均衡
   - 熔断器、限流与降级
3. Docker Compose 多容器编排
4. 应用案例与演示代码



1 云原生的概念与核心理念

- 什么是云原生？

01 云原生的定义（CNCF）

西南交通大学 Southwest Jiaotong University

CNCF（Cloud Native Computing Foundation）官方定义

云原生技术使组织能够在公有云、私有云和混合云等现代动态环境中，构建和运行可弹性扩展的应用。云原生的代表技术包括容器、服务网格、微服务、不可变基础设施和声明式API。这些技术能够构建容错性好、易于管理和便于观察的松耦合系统。结合可靠的自动化手段，云原生技术使工程师能够轻松地对系统进行频繁且可预测的重大变更。

CNCF简介:2015年成立

托管超过160+个云原生开源项目

代表项目：Kubernetes、Prometheus、Envoy、Helm、Istio

02 云原生技术生态全景

西南交通大学 Southwest Jiaotong University

生态规模：CNCF目前托管超过160+个开源项目，覆盖云原生的方方面面。

核心技术栈层级切片

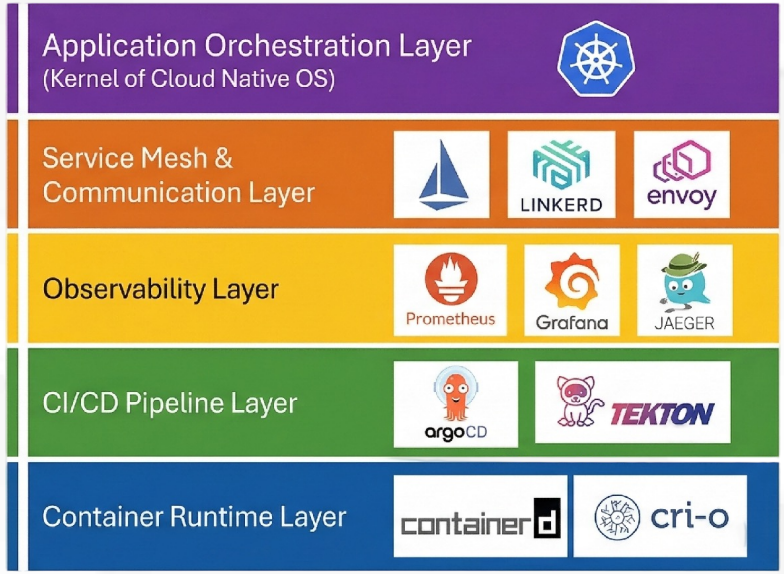
应用编排层：Kubernetes（绝对的行业标准，云原生操作系统的内核）。

微服务通信层：Istio, Linkerd（服务网格），Envoy（高性能代理）。

可观测性层：Prometheus（指标监控），Grafana（可视化），Jaeger（链路追踪）。

CI/CD流水线：ArgoCD（GitOps实践），Tekton。

容器运行时层：Containerd，CRI-O。



01 云原生的定义（CNCF）

西南交通大学 Southwest Jiaotong University

CNCF（Cloud Native Computing Foundation）官方定义

云原生技术使组织能够在公有云、私有云和混合云等现代动态环境中，构建和运行可弹性扩展的应用。云原生的代表技术包括容器、服务网格、微服务、不可变基础设施和声明式API。这些技术能够构建容错性好、易于管理和便于观察的松耦合系统。结合可靠的自动化手段，云原生技术使工程师能够轻松地对系统进行频繁且可预测的重大变更。

- **Cloud：**应用长在云上（云计算的五大本质）
  - 按需自服务、弹性扩展、资源池化、网络访问、按量计费
- **Native：**应用从写代码的第一天起，就是为了适应云环境而设计

01 云原生的核心要素

西南交通大学 Southwest Jiaotong University

动态环境：应用运行在多样的云基础设施上

弹性扩展：根据负载自动伸缩

容错性：系统能优雅地处理故障

松耦合：组件之间依赖最小化

自动化：减少人工干预，提高变更效率

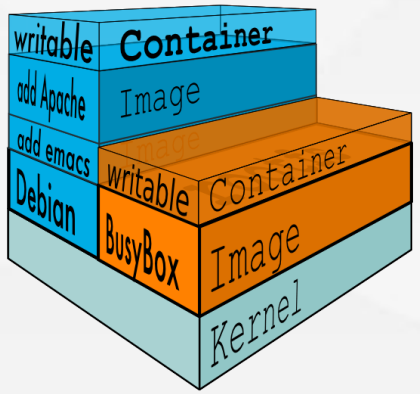
要素	定义	核心价值
容器化（Containerization）	应用及其依赖打包为容器镜像，确保环境一致性	可移植性、快速部署
微服务（Microservices）	将应用拆分为小型、独立的服务，每个服务专注于单一业务功能	独立开发、独立部署、独立扩展
声明式 API（Declarative APIs）	用户声明期望状态，系统自动实现和维护	简化运维、自动化管理
不可变基础设施（Immutable Infrastructure）	基础设施组件一旦部署不再修改，更新通过替换而非修补	环境一致性、可审计性

01 不可变基础设施

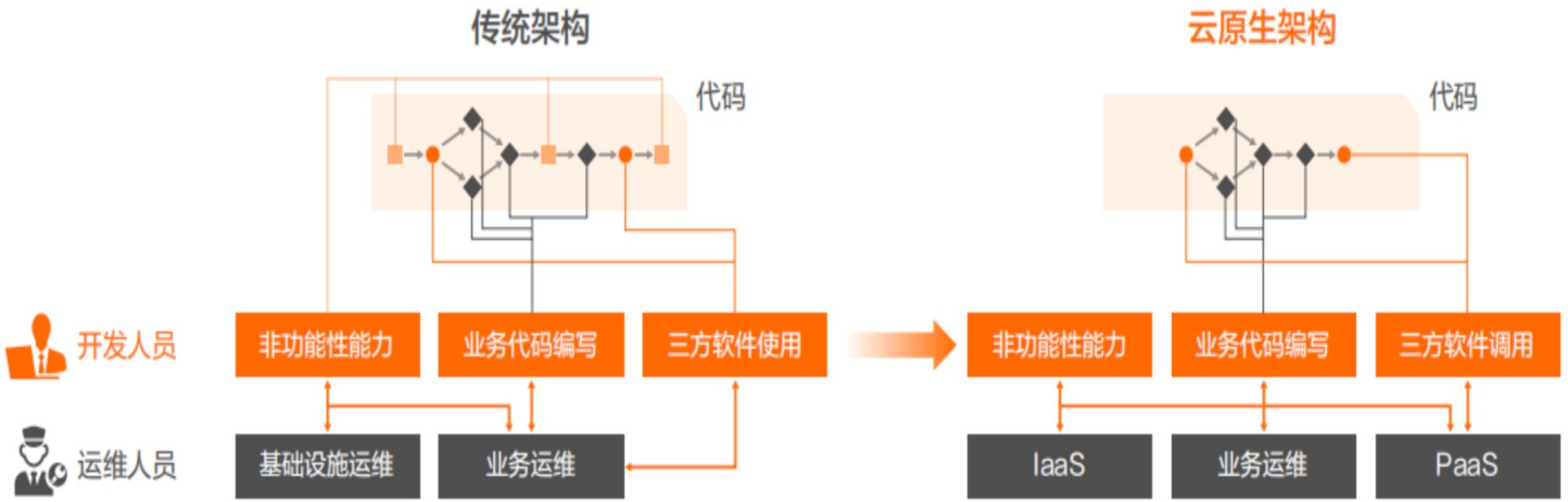
传统模式（可变）：在服务器上直接修改配置、安装补丁，易导致配置漂移，形成“雪花服务器”，使环境一致性难以保障。

云原生模式（不可变）：当需要变更时，通过构建新镜像并替换旧实例的方式完成更新，从根源上保证了环境的一致性与可重复性。

对比	可变基础设施	不可变基础设施
更新方式	SSH 登录服务器手动修改	构建新镜像，滚动替换
环境一致性	低（易出现配置漂移）	高（镜像统一）
回滚	困难	简单（直接使用旧版本镜像）
可审计性	弱	强（镜像版本化管理）



01 云原生 v.s. 传统开发模式



- 传统架构中，开发与运维需分别承担大量非功能与基础设施工作；
- 云原生架构则通过 IaaS/PaaS 平台解耦底层负担，让团队更聚焦于业务价值交付。

01 云原生 v.s. 传统开发模式

传统开发模式以单体架构、低频部署和人工运维为特征，而云原生模式通过微服务、容器化与自动化交付，实现了更敏捷、弹性和可靠的软件生命周期管理。

维度	传统开发模式	云原生模式
架构	单体应用（Monolith）	微服务架构
部署单元	WAR/EAR 包部署到应用服务器	容器镜像部署到 K8s
部署频率	每月 / 每季度	每天多次
扩展方式	垂直扩展（加 CPU / 内存）	水平扩展（加实例）
环境管理	手动配置、脚本	声明式配置（IaC）
故障处理	人工排查、手动恢复	自动检测、自动恢复
开发流程	瀑布式 / 长周期迭代	CI/CD、DevOps
基础设施	物理机 / 虚拟机	容器 / Serverless
状态管理	有状态（Session）	无状态（Stateless）

“这些技术能够构建容错性好、易于管理和便于观察的**松耦合系统**”
——CNCF定义

01 云原生 v.s. 传统开发模式

传统开发模式以单体架构、低频部署和人工运维为特征，而云原生模式通过微服务、容器化与自动化交付，实现了更敏捷、弹性和可靠的软件生命周期管理。

维度	传统开发模式	云原生模式
架构	单体应用（Monolith）	微服务架构
部署	WAR/EAR 包部署到应用服	
部署频率	每月 / 每季度	每天多次
扩展方式	垂直扩展（加 CPU / 内存）	水平扩展（加实例）
环境管理	手动配置、脚本	声明式配置（IaC）
故障处理	人工排查、手动恢复	自动检测、自动恢复
开发流程	瀑布式 / 长周期迭代	CI/CD、DevOps
基础设施	物理机 / 虚拟机	容器 / Serverless
状态管理	有状态（Session）	无状态（Stateless）

实现一个云原生应用，需要遵循什么规则？

“这些技术能够构建容错性好、易于管理和便于观察的松耦合系统”

02 十二要素应用（12Factor App）

十二要素应用方法论（The TwelveFactor App）是由Heroku联合创始人Adam Wiggins于2011年提出的，旨在为构建现代SaaS应用提供一组最佳实践指南。



要素	核心思想
基准代码（Codebase）	一份代码库对应多份部署
依赖（Dependencies）	显式声明并隔离依赖
配置（Config）	将配置存储在环境变量中
后端服务（Backing Services）	把后端服务当做附加资源
构建、发布、运行（Build, Release, Run）	严格分离构建和运行阶段
进程（Processes）	以无状态进程运行应用
端口绑定（Port Binding）	通过端口绑定提供服务
并发（Concurrency）	通过进程模型横向扩展
易处理（Disposability）	快速启动和优雅终止
开发 / 生产等价（Dev/Prod Parity）	尽可能保持开发、预发布、生产环境一致
日志（Logs）	把日志当做事件流
管理进程（Admin Processes）	后台管理任务作为一次性进程运行

02 十二要素详解（重点要素）

西南交通大学 Southwest Jiaotong University

因素3：配置（Config）
原则：将配置（数据库连接、API密钥、环境参数）存储在环境变量中，而非硬编码在代码里
好处：同一份代码可以在开发、测试、生产环境中运行，只需改变环境变量
云原生实践：Kubernetes ConfigMap、Docker环境变量

因素6：无状态进程（Stateless Processes）
原则：应用进程应该是无状态的，任何需要持久化的数据都存储在后端服务中（数据库、缓存、对象存储）
好处：任何实例都可以处理任何请求，方便水平扩展和故障恢复
反模式：Session Sticky（会话粘滞）——将用户会话绑定到特定实例

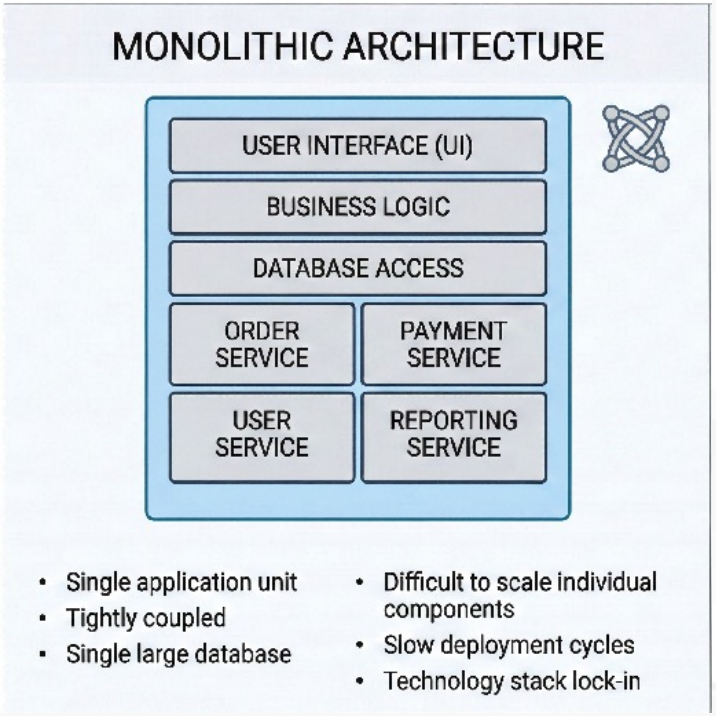
因素9：易处理性（Disposability）
原则：进程可以瞬间启动和优雅终止
快速启动：秒级启动，方便弹性扩缩容
优雅终止：收到SIGTERM后完成当前请求再退出，不丢数据
云原生实践：容器秒级启动；K8s preStop hook + terminationGracePeriodSeconds

因素10：开发/生产等价（Dev/Prod Parity）
原则：缩小开发、测试、生产环境的差距（时间、人员、工具）
实践：Docker确保环境一致；CI/CD自动化部署

2 微服务架构设计原则

- 从单体到微服务

02 单体（Monolith）架构



单体架构：

- 一个紧密耦合的整体
- 优点：开发快、部署简单
- 局限：难以独立扩展和更新迭代

02 微服务的定义与特征

西南交通大学 Southwest Jiaotong University

微服务架构（Microservices Architecture） 是一种将应用程序构建为一组小型、自治的服务的架构风格。每个服务围绕特定业务能力构建，独立部署，通过轻量级机制（通常是HTTP API）进行通信。

——Martin Fowler & James Lewis, 2014

02 微服务的核心特征

西南交通大学 Southwest Jiaotong University

微服务架构（Microservices Architecture） 是一种将应用程序构建为一组小型、自治的服务的架构风格。每个服务围绕特定业务能力构建，独立部署，通过轻量级机制（通常是HTTP API）进行通信。 ——Martin Fowler & James Lewis, 2014

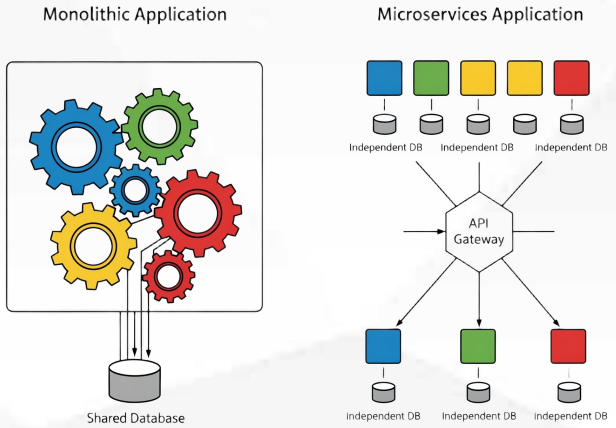
特征	说明
单一职责	**每个服务只负责一个明确的业务功能**
独立部署	**服务可以独立构建、测试和部署，不影响其他服务**
独立扩展	根据各服务的负载独立进行水平扩展
技术异构	不同服务可使用不同的编程语言和数据存储
轻量级通信	**服务间通过 REST API、gRPC 或消息队列通信**
去中心化治理	各团队自主选择技术栈和开发流程
去中心化数据	每个服务拥有自己的数据库（Database per Service）
容错设计	服务需要考虑依赖故障的处理（熔断、降级、重试）

02

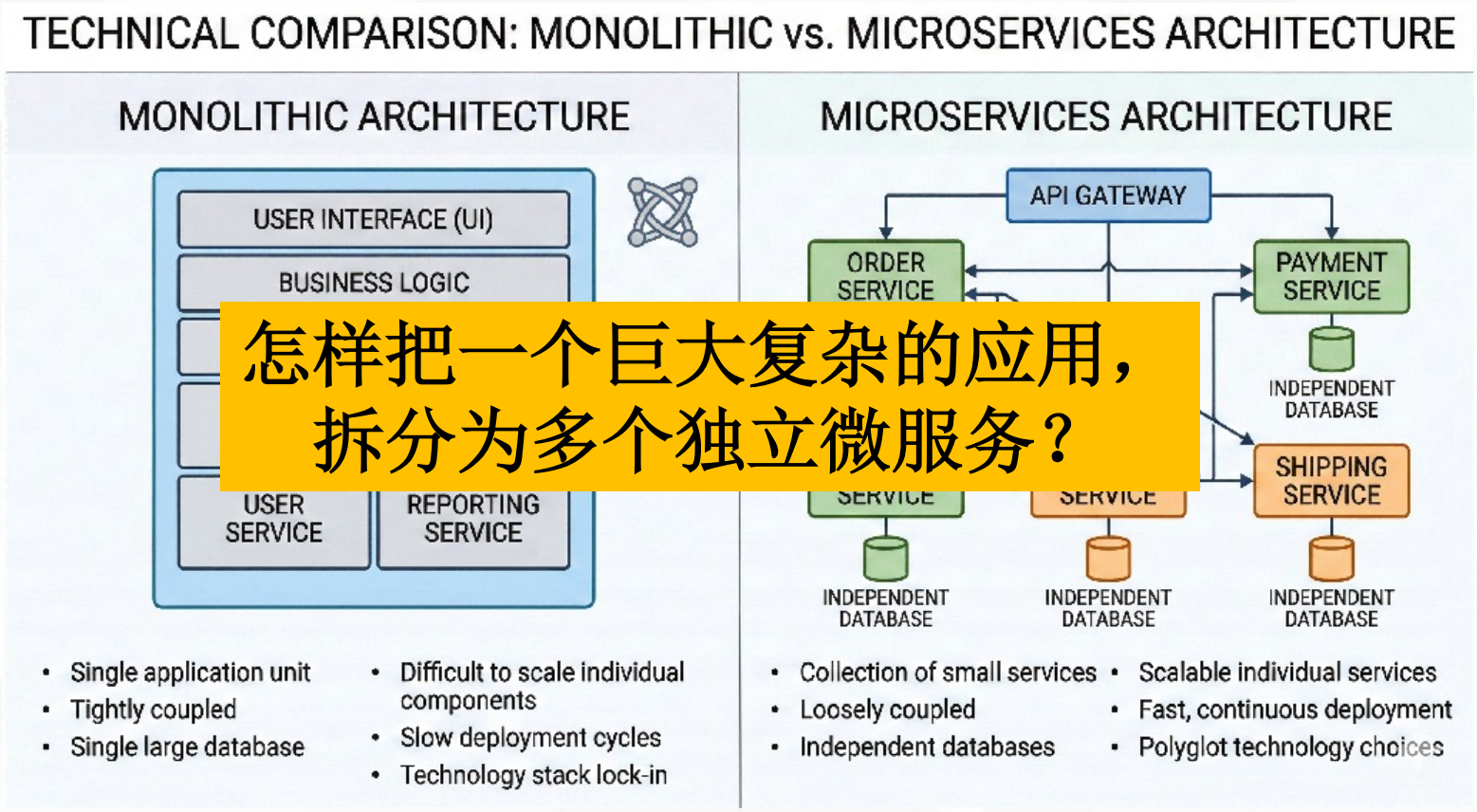
单体架构 vs 微服务架构

- 单体架构初期开发快、部署简单，但随着业务膨胀，扩展与维护成本会急剧升高；
- 微服务架构通过拆分与独立演进，虽初期基础设施投入更大，却能在长期迭代中保持高效与稳定，更适合大规模、高迭代的业务场景。

维度	单体架构（Monolith）	微服务架构
代码组织	所有功能在一个代码库中	每个服务独立代码库
部署	整体打包、整体部署	各服务独立部署
扩展	整体扩展（即使只有一个模块需要）	按需扩展特定服务
技术栈	统一技术栈	可混合使用多种语言 / 框架
数据库	共享数据库	每个服务独立数据库
团队协作	大团队协作，容易冲突	小团队（2 Pizza Team），职责明确
故障影响	一个模块故障可能导致整体崩溃	单个服务故障不影响整体
开发速度	初期快，后期急剧变慢	初期慢（基础设施复杂），后期持续高效



02 单体架构 vs 微服务架构



怎样把一个巨大复杂的应用，拆分为多个独立微服务？

02 微服务拆分策略

西南交通大学 Southwest Jiaotong University

如何确定服务边界？

1. 按业务领域拆分（DomainDriven Design, DDD）

- 基于**领域驱动设计**的**限界上下文（Bounded Context）**
- 每个限界上下文对应一个微服务
- 例：电商系统：用户服务、商品服务、订单服务、支付服务、库存服务

2. 按数据边界拆分

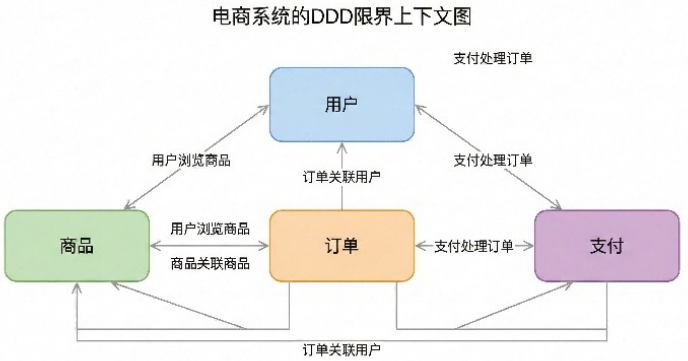
- 每个服务拥有自己的数据库
- 服务间不直接访问对方数据库
- 通过API或事件进行数据交换

3. 按变更频率拆分

- 变更频率高的功能独立为服务
- 稳定的功能保持在一起

02 拆分原则

原则	说明
高内聚	相关功能放在同一个服务内
低耦合	服务间依赖最小化
可独立部署	修改一个服务不需要重新部署其他服务
适当粒度	既不过粗（变成小单体）也不过细（网络开销大）



微服务拆分遵循高内聚、低耦合的核心原则，确保服务可独立部署，并通过适当粒度避免过小或过大的设计陷阱；在电商等场景中，可基于 DDD 限界上下文（如用户、商品、订单、支付）清晰划分服务边界。

服务拆分之后带来什么新问题？

02 服务拆分后的新痛点

西南交通大学 Southwest Jiaotong University

- 微服务之间如何通信？
- 如何定位到想要的微服务位置？
- 如果有微服务崩了怎么办？
- 怎么防止微服务崩溃？

03 微服务通信方式——同步通信

- REST：
  - 前端调用后端（对外API）
  - 基于 HTTP/JSON
  - 人类易读、简单通用、生态成熟、浏览器支持好
- gRPC：
  - 微服务A调用微服务B（对内API）
  - 基于 HTTP/2 和 Protocol buffer（二进制）
  - 体积小、传输快、更安全、适合微服务内部高性能通信

特性	REST (Representational State Transfer)	gRPC (Google Remote Procedure Call)
协议	HTTP/HTTPS	HTTP/2
数据格式	JSON（主流）、XML	Protocol Buffers（二进制，高效）
设计风格	资源导向（GET/POST/PUT/DELETE）	方法调用导向（RPC）
优点	简单、通用、生态成熟	高性能、强类型、双向流、多语言代码生成
缺点	文本协议，序列化开销大；缺乏类型安全	学习曲线较高，调试不如 REST 直观

03 微服务通信方式——异步通信

西南交通大学 Southwest Jiaotong University

消息队列（Message Queue）

异步通信以消息队列为核心，通过 Kafka、RabbitMQ 等消息中间件，将生产者发送的消息暂存并异步交付给消费者处理，实现服务间的解耦；它具备削峰填谷、保障最终一致性的优势，能有效提升系统的弹性与可用性，适用于事件通知、异步处理及跨服务数据同步等场景。

03 REST vs gRPC 详细对比

REST 基于 HTTP/1.1 和 JSON，接口定义灵活、生态成熟，是对外 API 和 Web 前端调用的首选；gRPC 基于 HTTP/2 和 Protobuf，凭借强类型、高性能与流式传输能力，更适合微服务内部的高性能通信场景，但需要 gRPC-Web 代理才能在浏览器中使用。

对比维度	REST	gRPC
传输协议	HTTP/1.1（文本）	HTTP/2（二进制帧）
序列化格式	JSON（文本，可读性好）	Protobuf（二进制制，体积小）
接口定义	OpenAPI/Swagger（可选）	.proto 文件（强制，IDL）
代码生成	可选	自动生成多语言客户端 / 服务端代码
流式传输	不原生支持	支持四种模式（Unary、Server/Client/Bidirectional Streaming）
性能	一般（JSON 解析开销）	高（Protobuf 体积比 JSON 小 310 倍）
浏览器支持	完美支持	需要 gRPCWeb 代理
生态	极其成熟	快速增长中
适用场景	对外 API、Web 前端调用	内部微服务间高性能通信

03 gRPC Protobuf 示例

西南交通大学 Southwest Jiaotong University

```
syntax = "proto3";
service UserService {
    rpc GetUser (GetUserRequest)
returns (User) {}
    rpc ListUsers (ListUsersRequest)
returns (stream User) {}
}
message GetUserRequest {
    string user_id = 1;
}
message User {
    string id = 1;
    string name = 2;
    string email = 3;
}
```

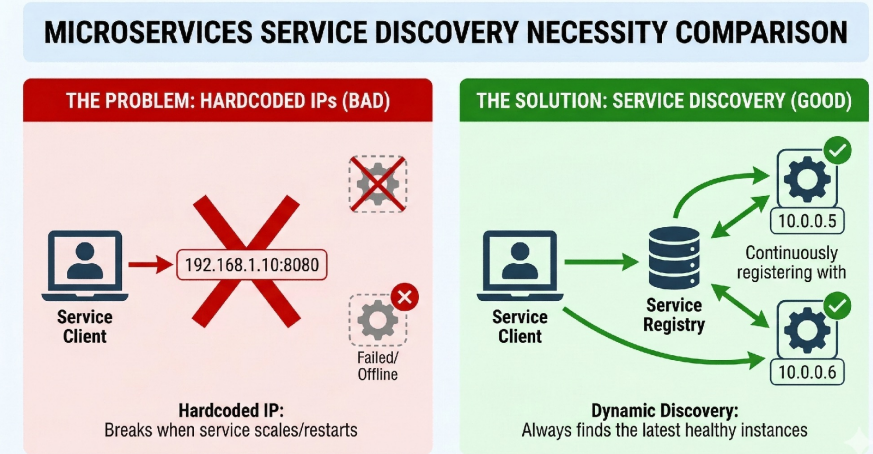
这段示例通过 .proto 文件明确定义了服务接口与数据结构，既支持普通的单请求 - 单响应调用，也支持服务端流式返回，是 gRPC 实现强类型、高性能服务通信的典型方式。

04 服务发现与注册

西南交通大学 Southwest Jiaotong University

为什么需要服务发现？

- 在单体时代，数据库IP是写死在配置文件里的
- 微服务架构中，服务实例的IP地址和端口动态变化
- 服务可能随时扩缩容、故障重启、迁移到新节点
- 硬编码地址行不通，需要动态发现服务位置



04 服务发现模式

西南交通大学 Southwest Jiaotong University

客户端发现（ClientSide Discovery）

服务 A 发起调用请求，经负载均衡器或 API 网关路由，查询服务注册中心获取可用实例，再将请求转发至服务 B 完成通信。

- 代表：Kubernetes Service + kubeproxy
- 优点：架构简单，无额外网络跳转，延迟低
- 缺点：客户端需集成服务发现与负载均衡逻辑，与语言耦合

04 服务发现模式

西南交通大学 Southwest Jiaotong University

服务端发现（Server-Side Discovery）

服务 A 直接向负载均衡器 / API 网关发起请求，由网关或负载均衡器查询服务注册中心，获取服务 B 的可用实例列表，再通过负载均衡策略将请求转发到目标实例完成通信。

- 代表：Nginx、Spring Cloud Gateway、Kubernetes Service + kube-proxy
- 优点：客户端逻辑简单，与开发语言无关，负载策略统一在网关侧维护
- 缺点：多了一层网络跳转，会引入额外延迟，且网关 / 负载均衡器本身需要高可用保障

对比维度	客户端发现（Client-Side Discovery）	服务端发现（Server-Side Discovery）
核心流程	服务 A 直接查询注册中心，获取服务 B 实例列表，本地负载均衡后直接发起调用	服务 A 将请求发送给负载均衡器 / API 网关，由网关查询注册中心并完成负载均衡，再转发到服务 B 实例
代表实现	Eureka + Ribbon、Consul + 客户端负载均衡	Nginx、Spring Cloud Gateway、Kubernetes Service + kube-proxy
优点	架构简单，无额外网络跳转，延迟低	客户端逻辑极简，语言无关，负载策略统一维护，便于管控
缺点	客户端需集成服务发现与负载均衡逻辑，与语言耦合	多一跳网络延迟，网关 / 负载均衡器成为潜在单点，需高可用保障

04 服务注册中心

主流服务注册中心在一致性模型与能力侧重上各有特点：

- etcd 与 Consul 均基于 Raft 实现强一致性，前者是 Kubernetes 的默认组件，专注键值存储；后者则内置了服务发现、健康检查等全栈能力。
- ZooKeeper 作为老牌组件，通过 ZAB 协议保证强一致性，在配置管理场景中应用广泛。
- Nacos 则支持 AP/CP 模式切换，由阿里开源，兼顾了服务发现与配置管理的双重需求，更适配云原生微服务架构。

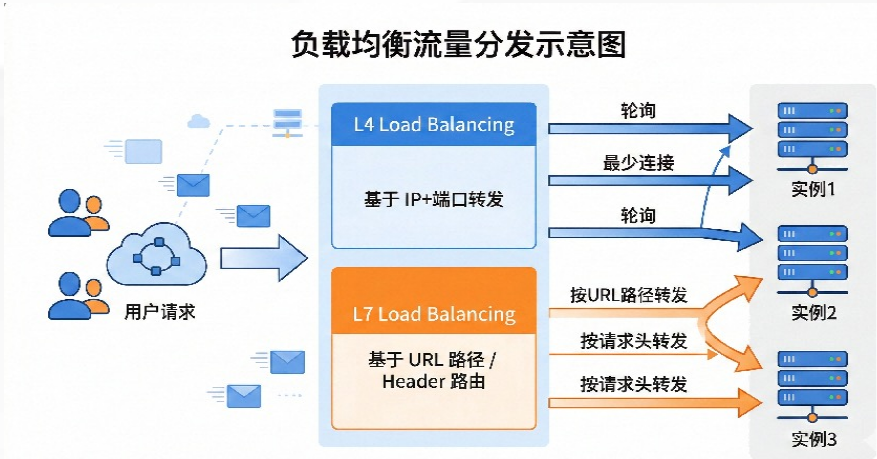
注册中心	一致性模型	特点
etcd	强一致（Raft）	K8s 默认使用，键值存储
Consul	强一致（Raft）	内置服务发现、健康检查、KV 存储
ZooKeeper	强一致（ZAB）	老牌，配置管理
Nacos	AP/CP 可切换	阿里开源，服务发现 + 配置管理

05 负载均衡策略

负载均衡（Load Balancing）是一种将网络流量或工作负载分发到多个服务实例上的技术，旨在提高可用性、提升性能和避免单点过载。

算法	原理	适用场景
轮询（Round Robin）	按顺序依次分发到每个实例	实例性能相同
加权轮询（Weighted RR）	根据实例权重（性能）按比例分发	实例性能不均
最少连接（Least Connections）	分发到当前连接数最少的实例	长连接场景
随机（Random）	随机选择实例	简单场景
一致性哈希（Consistent Hash）	相同请求哈希到相同实例	缓存命中率优化
IP 哈希	按客户端 IP 哈希选择实例	Session 亲和性

05 负载均衡层次



层次	技术	说明
L4负载均衡	TCP/UDP层	基于IP+端口转发，性能高
L7负载均衡	HTTP层	基于URL路径、Header等路由，功能丰富

- L4 负载均衡
  - TCP/UDP 层
  - 基于 IP 和端口进行转发
  - 速度快，只基于IP和端口
  - 轮询、最少连接等算法
- L7 负载均衡
  - HTTP 层
  - 智能路由：根据 URL 路径、请求头等
  - 功能更丰富灵活，满足复杂的业务需求

06 熔断器模式（Circuit Breaker）

西南交通大学 Southwest Jiaotong University

问题：级联故障

在微服务架构中，服务A调用服务B，服务B调用服务C。

如果服务C崩溃：

- 服务B等待C响应 → 线程阻塞 → 线程池耗尽
- 服务A等待B响应 → 同样阻塞
- **雪崩效应（Cascading Failure）**：一个服务故障导致整个链路崩溃

06 熔断器模式原理

西南交通大学 Southwest Jiaotong University

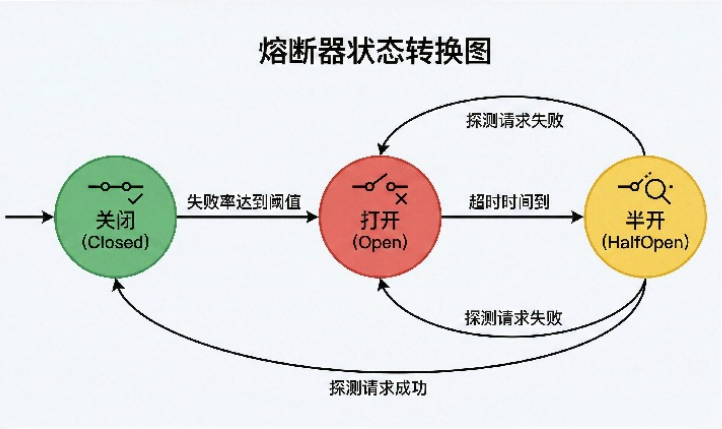
熔断器模式（Circuit Breaker Pattern） 受电路断路器启发。当检测到下游服务故障达到阈值时，自动"断开"对该服务的调用，直接返回降级响应，防止故障扩散。

- 熔断器模式的三种状态
  - 关闭（closed）：正常通信
  - 打开（opened）：触发熔断机制，返回fast-fail，停止调用下游微服务
  - 半开（halfopen）：尝试通过少量请求，探测下游服务是否恢复



状态	说明	行为
关闭（Closed）	正常状态	所有请求正常转发；统计失败率
打开（Open）	熔断触发	所有请求直接返回降级响应；不调用下游
半开（HalfOpen）	尝试恢复	允许少量请求通过；成功则关闭熔断，失败则继续打开

06 状态转换



熔断器通过三种核心状态的切换，实现对服务调用的保护：

Closed（关闭状态）

这是熔断器的常态，服务调用正常进行。当调用失败率达到预设阈值时，熔断器会立即切换到 Open（打开） 状态，快速熔断，防止故障扩散。

Open（打开状态）

此时，所有对目标服务的调用都会被直接拒绝，并快速返回错误。经过一段超时时间后，熔断器会自动进入 HalfOpen（半开） 状态，试探性地检查服务是否恢复。

HalfOpen（半开状态）

熔断器会允许少量探测请求通过。

如果探测请求成功，说明服务已恢复，熔断器将恢复到 Closed 状态。

如果探测请求失败，说明服务仍不可用，熔断器将回到 Open 状态，继续保护调用方。

06 熔断器实现与实践——关键参数

熔断器的核心参数共同定义了其容错行为：

- 失败率阈值：触发熔断的失败比例，典型值为 50%，即失败请求占比过半时熔断。
- 统计窗口：用于统计失败率的时间范围，通常为 10 秒，确保判断基于近期状态。
- 最少请求数：触发熔断所需的最小请求量，如 20 次，避免少量波动误判。
- 熔断持续时间：Open 状态的持续时长，如 30 秒，给服务恢复留出时间。
- 半开允许请求数：HalfOpen 状态下允许通过的探测请求数，如 5 次，用于验证服务是否恢复。
- 这些参数协同工作，让熔断器既能快速响应故障，又能稳妥地尝试恢复服务。

参数	说明	典型值
失败率阈值	触发熔断的失败比例	50%
统计窗口	统计失败率的时间窗口	10秒
最少请求数	触发熔断需要的最少请求量	20次
熔断持续时间	Open状态持续时间	30秒
半开允许请求数	HalfOpen状态允许通过的请求数	5次

06 与其他容错模式配合

西南交通大学 Southwest Jiaotong University

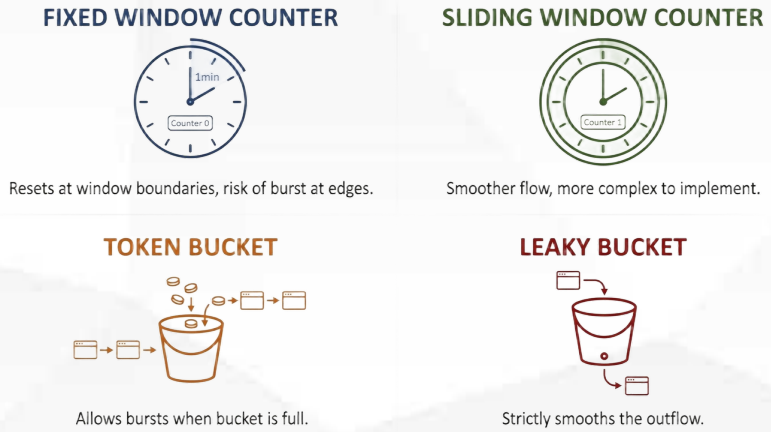
- **重试（Retry）**：短暂故障时自动重试（配合指数退避）
- **超时（Timeout）**：设置调用超时上限，防止长时间阻塞
- **舱壁（Bulkhead）**：资源隔离，不同服务使用独立的线程池/连接池
- **降级（Fallback）**：熔断触发后返回默认值或缓存数据

07 限流与降级策略

西南交通大学 Southwest Jiaotong University

限流（Rate Limiting）：通过限制单位时间内的请求数量来保护系统免受过载

常见限流算法		
算法	原理	特点
固定窗口计数器	时间窗口内计数，超过阈值拒绝	简单，但有窗口边界突刺问题
滑动窗口计数器	滑动时间窗口，平滑限流	更平滑，实现稍复杂
令牌桶（Token Bucket）	固定速率生成令牌，请求消耗令牌	允许突发流量（桶满时）
漏桶（Leaky Bucket）	请求进入桶，以固定速率流出	严格平滑流量



07 降级策略（Degradation）

降级：当系统压力过大或依赖服务故障时，主动降低服务质量以保证核心功能可用

策略	说明	示例
返回默认值	返回预设的默认响应	推荐服务降级 → 返回热门商品
返回缓存	返回上次成功的缓存数据	用户画像服务降级 → 返回缓存画像
功能裁剪	关闭非核心功能	高峰期关闭评论、推荐功能
流量排队	请求排队等待处理	秒杀场景排队机制

课程群



西南交通大学
Southwest Jiaotong University

02 服务拆分后的新痛点

西南交通大学 Southwest Jiaotong University

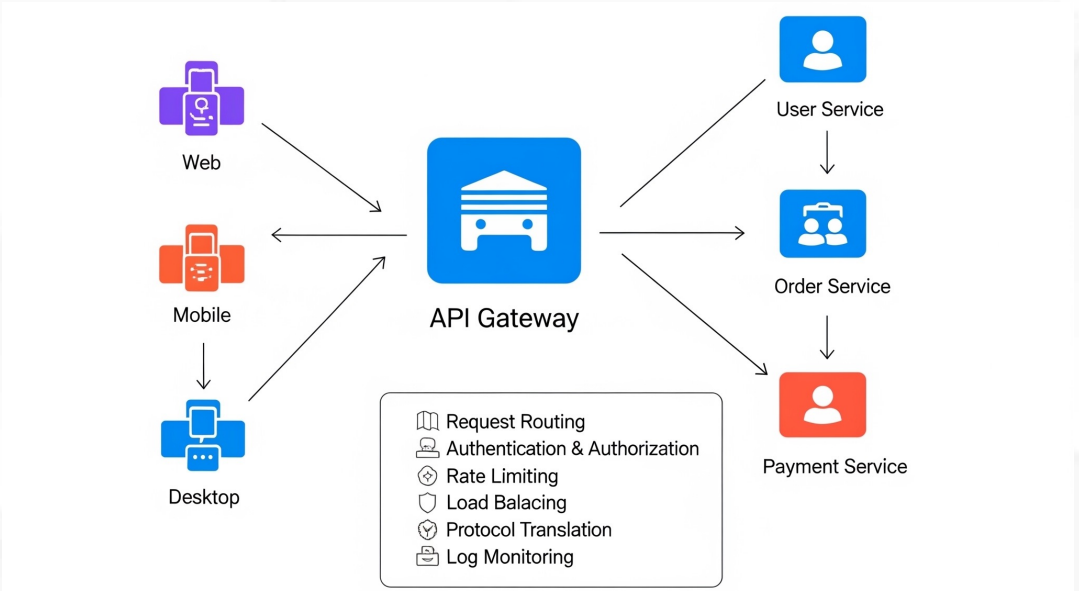
- 微服务之间如何通信？
- 如何定位到想要的微服务位置？
- 如果有微服务崩了怎么办？
- 怎么防止微服务崩溃？

上述功能具体在哪里部署实现？

08 API网关

- API网关（API Gateway） 是微服务架构中的统一入口，负责接收所有客户端请求，进行路由、认证、限流、协议转换等处理后，将请求转发到对应的后端微服务。

功能	说明
请求路由	根据URL路径将请求路由到对应微服务
认证鉴权	统一的身份验证和权限校验
限流熔断	保护后端服务免受过载
负载均衡	将请求分发到多个服务实例
协议转换	外部REST → 内部gRPC
请求聚合	将多个微服务的响应聚合为一个
日志监控	统一的请求日志和指标采集



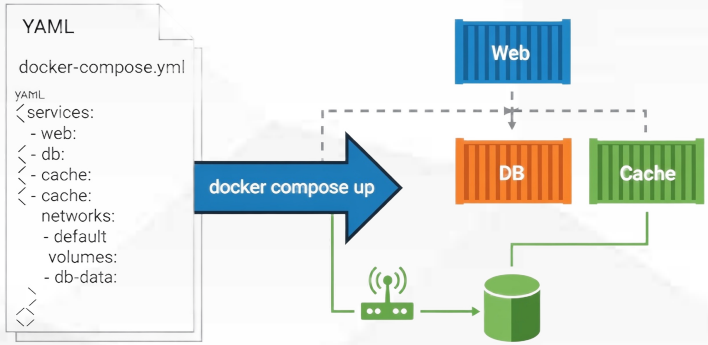
3 Docker Compose 多容器编排

- 使用Compose管理微服务

03 Docker Compose 核心概念

- Docker Compose 是一个用于定义和运行多容器Docker应用的工具
  - 使用YAML文件描述应用的服务、网络和卷
  - 一次声明定义，一键拉起整个集群
  - ~~Docker run~~

    Docker –compose up



Define once, run anywhere

03 Compose文件核心结构

顶级key	说明
services	定义应用包含的服务（容器）
networks	定义自定义网络
volumes	定义持久化卷
configs	定义配置文件
secrets	定义敏感信息

Compose 文件的核心结构由五大顶级 key 构成，它们各司其职，共同定义了整个多容器应用的编排规范：

- **services**：定义应用所包含的所有服务（容器），是整个文件的核心，用于配置每个服务的镜像、端口、环境变量等。
- **networks**：定义应用所需的自定义网络，实现服务间的隔离与通信，确保容器在安全可控的网络环境中交互。
- **volumes**：定义数据持久化卷，用于保存容器运行时产生的数据，避免容器删除后数据丢失。
- **configs**：定义配置文件，将配置与镜像解耦，方便在不重建镜像的情况下更新应用配置。
- **secrets**：定义敏感信息（如密码、密钥），通过安全的方式管理和传递敏感数据，提升应用的安全性。

这些顶级 key 协同工作，让复杂的多容器应用部署变得清晰、可维护且可复现。

03 服务（Service）配置选项

选项	说明	示例
image	使用已有镜像	image: nginx:alpine
build	从Dockerfile构建	build: ./api
ports	端口映射	ports: ["8080:80"]
environment	环境变量	environment: [DB_HOST=db]
volumes	数据卷挂载	volumes: [./data:/app/data]
depends_on	服务依赖关系	depends_on: [db, redis]
networks	加入的网络	networks: [frontend, backend]
deploy	部署配置	副本数、资源限制
healthcheck	健康检查	命令、间隔、超时

Docker Compose 中 services 作为核心配置模块，通过九大高频选项实现对容器服务的全维度定义，覆盖镜像获取、构建、网络、数据、依赖及运维管控等核心场景：

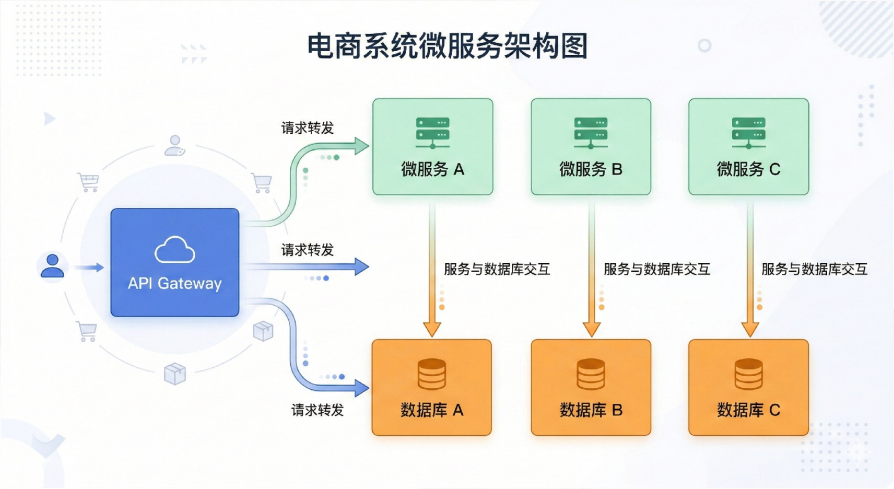
镜像与构建：image直接指定已有镜像快速启动容器，build则通过本地Dockerfile自行构建镜像，灵活适配不同部署需求；

网络与端口：ports实现容器端口与宿主机的映射，networks 为服务分配自定义网络，保障服务间通信的隔离性与可控性；

数据与环境：volumes挂载数据卷实现持久化存储，environment 配置服务运行所需的环境变量，实现配置与镜像的解耦；

依赖与运维：depends_on定义服务启动的依赖顺序，deploy配置副本数、资源限制等部署策略，healthcheck则通过自定义规则监控服务健康状态，确保容器稳定运行。

03 Docker Compose 微服务示例



典型的电商微服务系统架构：

统一入口：所有用户请求首先到达最左侧的API Gateway。它作为整个系统的唯一入口，负责请求路由、认证授权和流量管控，简化了客户端的调用逻辑。

服务拆分：网关将请求转发到中间层的三个核心微服务（如用户服务、订单服务、商品服务）。每个微服务都是独立部署、可独立扩展的业务单元，实现了系统的解耦。

数据隔离：每个微服务都拥有自己独立的数据库，这是微服务架构的关键设计原则。它确保了服务间的数据隔离，避免了单点故障的影响，同时也提升了数据访问的性能。

03 Compose 网络与服务发现

西南交通大学 Southwest Jiaotong University

默认网络行为

- Compose会自动创建一个**默认网络**（项目名_default）
- 同一网络中的容器可以通过**服务名**直接通信（Docker内置DNS）
- 例：userservice容器可以通过 userdb:5432 访问数据库

在 Docker Compose 环境中，网络与服务发现机制为微服务间的通信提供了极大的便利：

- 自动网络管理：Compose 会为每个项目自动创建一个默认网络（命名规则为 项目名_default），所有服务默认加入此网络。
- **内置 DNS 服务发现：**在同一网络内，容器之间无需知晓对方的 IP 地址，可直接通过服务名进行通信。Docker 内置的 DNS 服务器会自动将服务名解析为对应容器的内部 IP。
- 便捷通信示例：例如，userservice 容器可以直接通过 userdb:5432 这样的地址访问数据库服务，无需手动配置 IP，大大简化了服务间的调用配置。

03 自定义网络隔离

西南交通大学 Southwest Jiaotong University

```
networks:
  frontend:     # 外部可访问
    driver: bridge
  backend:      # 内部服务网络
    driver: bridge
    internal: true   # 禁止外部访问
```

在 Docker Compose 中，通过自定义网络可以实现精细化的服务隔离与安全管控：

frontend 网络：采用 bridge 驱动，允许外部访问，作为 API Gateway 与外部用户交互的入口网络。

backend 网络：同样采用 bridge 驱动，但通过 internal: true 配置禁止外部访问，专门用于微服务之间的内部通信，确保核心业务数据和服务对外部不可见。

这种双层网络设计，既保障了前端服务的对外可达性，又实现了内部服务的安全隔离，有效提升了系统的整体安全性。

4 应用案例

- 微服务在企业中的实践

03 应用案例一：亚马逊的微服务转型

西南交通大学 Southwest Jiaotong University

案例背景

•2001年前：亚马逊是典型的单体架构（Monolith）

•代码库庞大，开发团队间协作困难

•每次发布需要全公司协调，周期长、风险高

年份	事件
2001	Jeff Bezos发布著名的"API命令"：所有团队的服务必须通过API通信，不得直接访问他人数据
2002–2006	逐步将单体拆分为独立的微服务
2006	基于内部服务化实践推出AWS（S3、EC2）
至今	亚马逊运行数千个微服务，每天部署超过100,000次

03 Bezos的"API命令"核心原则

西南交通大学 Southwest Jiaotong University

- 所有团队必须通过服务接口暴露数据和功能
- 团队间的通信只能通过这些接口
- 不允许直接链接、直接读取其他团队的数据、共享内存
- 接口的具体技术不做规定（HTTP、CORBA、Pub/Sub等）
- 所有接口都必须从头设计为可对外暴露的
- 不遵守的人会被开除

启示

- 微服务转型是**组织架构变革**，不仅是技术变革
- 康威定律（Conway's Law）：系统架构反映组织结构

03 应用案例二：某互联网教育平台微服务实践

西南交通大学 Southwest Jiaotong University

案例背景

- 在线教育平台，支持直播课、录播课、作业系统、支付系统
- 初期单体架构，快速增长后遇到瓶颈
- 发布一次耗时2小时（全量回归测试）
- 直播高峰导致整个系统变慢（CPU全被直播模块占满）
- 新功能开发受限，团队间代码冲突频繁

03 微服务拆分方案

西南交通大学 Southwest Jiaotong University

服务	职责	技术栈	数据库
用户服务	注册登录、个人信息	Python Flask	PostgreSQL
课程服务	课程管理、课件上传	Python Django	PostgreSQL
直播服务	实时音视频推流	Go	Redis
作业服务	作业布置、批改	Python FastAPI	MongoDB
支付服务	支付和退款	Java Spring Boot	MySQL
通知服务	消息推送、邮件	Node.js	Redis

用户服务：负责注册登录与个人信息管理，采用轻量的 Python Flask 框架，搭配 PostgreSQL 保证关系型数据的一致性。

课程服务：承担课程管理与课件上传，使用 Python Django 快速构建后台管理功能，数据同样存储在 PostgreSQL。

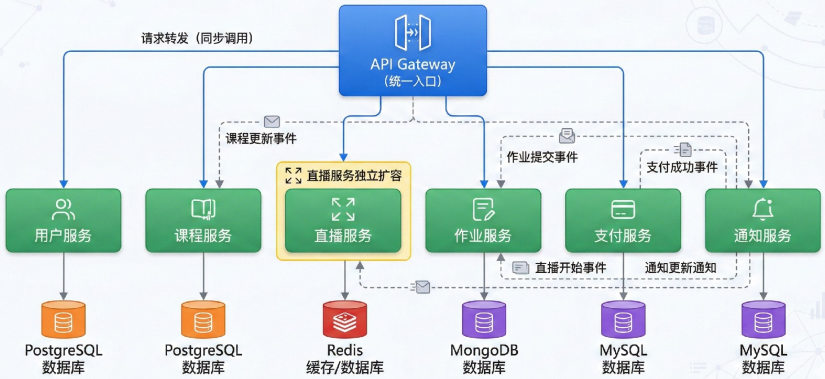
直播服务：专注实时音视频推流，选用高性能的 Go 语言，配合 Redis 实现低延迟的直播状态管理。

作业服务：处理作业布置与批改，采用 Python FastAPI 构建高性能接口，使用 MongoDB 存储灵活的作业数据。

支付服务：处理支付与退款，选用 Java Spring Boot 保证金融级别的稳定性和安全性，数据存储在 MySQL。

通知服务：负责消息推送与邮件发送，使用 Node.js 实现异步事件处理，通过 Redis 缓存消息队列。

03 架构改造效果



指标	改造前	改造后
发布频率	每周1次	每天35次
发布时间	2小时	10分钟
直播高峰响应	全系统变慢	仅直播服务扩容，其他服务不受影响
新功能交付周期	4周	12周

在通信方式上，网关与服务间采用同步调用保障实时响应，服务间通过异步通信实现解耦联动。这种架构直接支撑了右侧的核心指标提升：

发布效率：从每周 1 次提升到每天 35 次，发布时间从 2 小时缩短至 10 分钟，因为我们可以对 6 个服务进行独立部署，不再牵一发而动全身。

性能弹性：直播高峰时，我们只需对直播服务单独扩容，其他服务不受任何影响，彻底解决了改造前全系统变慢的问题。

交付速度：新功能交付周期从 4 周压缩到 12 周，独立的服务让多团队可以并行开发，极大提升了迭代效率。

03 微服务架构的挑战与应对

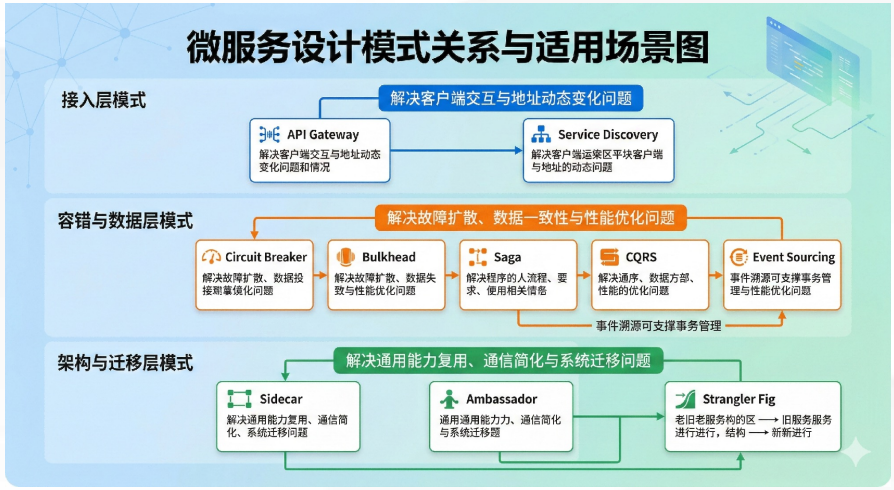
西南交通大学 Southwest Jiaotong University

挑战	说明	应对方案
分布式复杂性	网络延迟、部分故障、数据一致性	熔断器、重试、Saga模式
运维复杂度	几十上百个服务需要部署和监控	Kubernetes编排、CI/CD自动化
调试困难	跨服务调用链路追踪复杂	分布式追踪（Jaeger、Zipkin）
数据一致性	分布式事务处理困难	最终一致性、事件驱动
服务间通信	网络调用比进程内调用慢得多	gRPC高性能通信、异步消息
测试复杂度	端到端测试涉及多个服务	契约测试（Pact）、服务虚拟化
组织变革	需要配套的团队结构和文化	DevOps文化、小团队自治

03 微服务适用条件

- 团队规模大（>20人），需要并行开发
- 系统复杂度高，不同模块变化频率差异大
- 有成熟的基础设施（CI/CD、容器编排、监控）
- 小团队/简单项目——单体架构更合适
- 缺乏DevOps能力——运维成本可能高于收益

03 微服务设计模式总结



模式	说明	解决的问题
API Gateway	统一入口，路由/认证/限流	客户端与多服务的交互复杂性
Service Discovery	动态发现服务实例	服务地址动态变化
Circuit Breaker	熔断防止级联故障	下游服务故障扩散
Bulkhead	资源隔离（线程池/连接池）	共享资源被耗尽
Saga	分布式事务管理	跨服务数据一致性
CQRS	读写分离	读写性能优化
Event Sourcing	事件作为数据源	审计追踪、时间旅行
Sidecar	辅助容器提供横切功能	日志、监控、安全等通用能力
Ambassador	代理容器处理远程连接	简化服务间通信配置
Strangler Fig	逐步替换遗留系统	单体到微服务平滑迁移

03 演示代码说明

西南交通大学 Southwest Jiaotong University

运行环境：Jupyter Notebook（Python 3.8+）

演示代码 1：Docker Compose YAML生成器

- 使用Python程序自动生成Docker Compose配置文件
- 用户指定服务列表（名称、镜像、端口、依赖）
- 自动生成规范的 dockercompose.yml
- 包含网络隔离和数据卷配置

演示代码 2：Python实现熔断器模式

- 纯Python实现完整的Circuit Breaker
- 三种状态（Closed/Open/HalfOpen）的状态转换
- 模拟下游服务故障场景
- 展示熔断器如何防止级联故障
- 包含失败率统计和自动恢复

03 课堂互动与思考题

西南交通大学 Southwest Jiaotong University

思考题

1.架构决策：你正在开发一个大学选课系统（约5000用户），你会选择单体架构还是微服务架构？说明理由。

2.服务拆分：对于一个外卖平台（类似美团），请设计微服务拆分方案，列出至少5个核心微服务及其职责。

3.容错设计：如果推荐服务（非核心）崩溃了，订单服务（核心）应该如何处理？请设计降级策略。

4.通信选择：在什么场景下你会选择gRPC而非REST？在什么场景下REST更合适？

课后实验

1、使用Docker Compose编排一个包含3个服务的微应用：Web前端 + API后端 + PostgreSQL数据库

2、修改本讲的熔断器代码，增加"失败率"统计（而非简单计数），实现基于时间窗口的失败率熔断

03 本讲知识要点回顾

本讲围绕云原生与微服务架构的核心体系展开，核心要点可归纳为九大模块：

云原生定义：锚定 CNCF 官方定义，掌握容器化、微服务、声明式 API、不可变基础设施四大核心要素，建立云原生技术体系的底层认知。

12Factor App：以配置环境化、无状态进程、快速启停、开发 / 生产等价为核心，明确云原生应用的标准化开发准则。

微服务核心：紧扣单一职责、独立部署扩展、技术异构、去中心化数据四大特性，理解微服务架构的设计本质。

服务拆分：依托 DDD 限界上下文，结合数据边界与业务变更频率，掌握科学拆分微服务的核心方法论。

通信方式：按需选择通信方案 ——REST 用于通用场景，gRPC 适配高性能需求，消息队列实现服务间异步解耦。

服务发现：厘清客户端发现与服务端发现两种模式，熟悉 etcd、Consul、Nacos 等主流注册中心的应用。

容错模式：掌握熔断器、限流、降级、Saga 等核心容错手段，构建高可用的微服务弹性体系。

Compose 实践：精通 Docker Compose 在多容器编排、网络隔离、健康检查、服务发现等场景的落地配置，夯实微服务本地部署基础。

模块	核心知识点
云原生定义	CNCF定义、四大要素（容器化、微服务、声明式API、不可变基础设施）
12Factor App	配置环境化、无状态进程、快速启停、开发/生产等价
微服务	单一职责、独立部署/扩展、技术异构、去中心化数据
服务拆分	DDD限界上下文、数据边界、变更频率
通信方式	REST（通用）、gRPC（高性能）、消息队列（异步解耦）
服务发现	客户端发现 vs 服务端发现；注册中心（etcd/Consul/Nacos）
容错模式	熔断器（Circuit Breaker）、限流、降级、Saga
Compose	多容器编排、网络隔离、健康检查、服务发现

03 下次课预告 & Q&A

西南交通大学 Southwest Jiaotong University

下次课：第4讲 Kubernetes基础

- Kubernetes架构（Master/Worker节点组件）
- Pod概念与生命周期
- Service与服务发现
- Deployment与部署策略

预习建议：

- 安装Minikube或Kind，搭建本地K8s集群
- 了解YAML语法基础

03 课后作业

西南交通大学 Southwest Jiaotong University

课后作业

1.用自己的话解释云原生的四大核心要素，并各举一个实际技术/工具示例

2.设计一个在线书店的微服务架构：列出服务划分、通信方式、数据库选型

3.编写Docker Compose文件，编排一个"Flask API + Redis缓存 + PostgreSQL数据库"的三层应用